# What Does a Streaming Client See When an LLM Server Crashes?

A Fault-Injection Study of Client-Visible Token Anomalies in vLLM

Rajshekar Medipally

Independent researcher, Richmond, VA

https://github.com/rmedipallycic/llm-inference-fault-tolerance

Draft, October 2026

## Abstract

LLM inference servers stream generated tokens to clients over requests that last seconds to minutes. When a serving process fails mid-generation, the client is left holding a partial response, and the recovery it performs determines what it ultimately shows a user or hands to a downstream agent. We built an open-source fault-injection harness that SIGKILLs an unmodified OpenAI-compatible server at a chosen token position, restarts it, applies a client recovery strategy, and records every streamed chunk, and applied it to vLLM serving Llama 3.1 8B Instruct on one A10G GPU (60 trials, plus a 20-trial ablation). The central finding is that, with default settings, the failure-free output was not unique: at temperature 0, with requests issued one at a time, the first request after a server start and the second request with the identical prompt diverged at character 506 of about 1,280, and with a fixed seed at temperature 0.8 they diverged at character 9. Both outputs were exactly repeatable. Because a crashed server restarts cold, naive retry returned the first-request text in 30 of 30 trials and so contradicted text the client had already displayed in 25 of 30. Recompute, which resubmits the delivered text, never contradicted it, but its final text matched neither failure-free output in 24 of 30 trials. Recovery latency was server restart (median 117–178 s per strategy); the time to the first recovered token after restart was 0.23–0.32 s. Disabling vLLM's automatic prefix cache made the failure-free output reproducible (4 of 4 runs) and removed every contradiction (0 of 10 naive-retry trials), but recompute at temperature 0 still changed the answer, 43 characters after the kill, in 5 of 5 trials. All raw logs and the table-generating script are public.

## 1 Introduction

LLM serving has become stateful streaming infrastructure. A generation request holds a growing key-value (KV) cache on an accelerator and emits tokens to the client incrementally. Failure handling, however, is still largely stateless: if the serving process dies, the client or a load balancer resubmits the request and generation starts again.

That recovery has a cost that server-side metrics miss. A client that has already displayed 128 tokens may receive those tokens a second time, may receive a continuation that disagrees with what it already showed, or may lose text at the boundary. For a chat interface this is a visible glitch. For an agent that has already acted on a partial tool call or a partial plan, it is a correctness problem. Stream processing systems name this property *delivery semantics* and engineer exactly-once output deliberately [5, 2, 1]. LLM serving has no equivalent definition, and to our knowledge no measurement of what clients actually receive.

A common intuition is that at temperature 0 a retry is harmless: greedy decoding is deterministic, so resubmitting the prompt reproduces the same text and the client only has to drop the duplicate. Our measurements on vLLM refute that intuition. The engine's output for a fixed prompt depended on what the server had served earlier, so the restarted server produced a different, equally repeatable, text.

This paper makes three contributions:

1. **Definitions.** Token-level delivery semantics for streamed generation, and client-visible anomalies defined against *two* failure-free references rather than one (Section 3).

2. **A fault-injection harness** for any OpenAI-compatible server, which kills the full server process tree at a chosen chunk, restarts it, applies a client recovery strategy, and records every chunk with timestamps, the code commit, package versions, and GPU inventory (Section 4).

3. **A measurement study** of naive retry and recompute on vLLM at temperature 0 and 0.8 and three kill positions (Sections 5–6), which finds that failure-free output depends on server history and quantifies what that does to each recovery strategy.

The study deliberately uses unmodified engines and client-side recovery only. It establishes the baseline that a server-side exactly-once protocol, the next stage of this project, would have to improve on.

## 2 Background

**Autoregressive serving.** A request runs a prefill phase, which processes the prompt in parallel and builds the KV cache, followed by a decode phase that produces one token per step and streams it to the client. Engines such as vLLM [7] and SGLang [13] batch requests at iteration granularity [12] and manage the KV cache in fixed-size blocks.

**The state lost on failure.** The KV cache is a deterministic function of the weights and the token sequence, so it can always be rebuilt by prefill. What cannot be rebuilt from the prompt alone is the *sampling path*: at temperature above zero, the continuation depends on random draws whose state lives in the dead process.

**Rollback-recovery vocabulary.** Classic rollback-recovery distinguishes checkpointing from logging and defines *output commit*: a message to the outside world must not be released until the state that produced it is recoverable [5]. Mapped onto LLM serving, the token sequence is a replayable input log, the KV cache is derived operator state, and the streaming client is the external sink where delivery semantics are decided.

## 3 Delivery model and anomalies

A failure occurs after the client has received delivered text $D$. After recovery the client receives recovery text $R$. What the client finally holds depends on the strategy: **naive retry** resubmits the original prompt with the original `max_tokens`, as many client retry wrappers do, and the retry is a complete answer, so the final text is $F = R$; **recompute** resubmits the prompt concatenated with $D$ and requests the remaining token budget, so the server rebuilds the KV cache by prefill and continues, and $F = D \cdot R$. Neither requires engine changes.

**Two references.** Before the trials of each run, the harness issues the request twice without failures and records both outputs: $G$, the first request after the server starts, and $G'$, the second. A reproducible server has $G = G'$. We do not assume this; we measure it, and compare every final text with both.

**Anomalies.**

- **Contradiction** (naive retry): $R$ does not start with $D$, i.e. the replay disagrees with text the client has already displayed. A client that deduplicates by dropping the first $|D|$ characters of $R$ would then splice two different texts. Recompute cannot contradict $D$ by construction.
- **Duplication** (naive retry): the client receives $D$ again. Without contradiction this is full duplication, which an idempotent client can remove.
- **Divergence**: $F$ equals neither $G$ nor $G'$, i.e. the failure changed the final answer relative to any failure-free run we observed.
- **Recovery latency**: wall time from the kill to the first chunk delivered after recovery, which includes server restart.

Comparison is on text, not token ids, because the OpenAI-compatible streaming API does not guarantee one token per chunk.

## 4 Fault-injection harness

The harness (`harness/`, about 340 lines of Python) has three parts.

**Server lifecycle.** The server command is started in its own process group and declared ready when `GET /v1/models` returns 200. A fault is a `SIGKILL` of the whole process group, since vLLM and SGLang spawn worker processes that otherwise keep holding GPU memory and the port. On Windows, used only for laptop smoke tests, the equivalent is `taskkill /F /T`.

**Streaming client.** Requests go to `/v1/completions` with `stream=true`. Every server-sent event with non-empty text is recorded as a chunk with wall-clock and monotonic timestamps. The kill fires from the chunk callback when the configured chunk count is reached, and the client keeps reading, so chunks the server had already sent before dying are recorded as in-flight rather than lost.

**Trial protocol.** Each run (one engine, strategy, temperature, and kill position) starts the server, records two failure-free references $G$ and $G'$, then for each trial: streams the request, kills at the kill point, restarts the server and waits for readiness, issues the recovery request, and writes a JSON record containing every chunk, the kill time, the restart time, and the client view $V$.

**Provenance.** Each run records the git commit and whether the working tree was dirty, the full command line, installed versions of `vllm`, `sglang`, `torch`, and `transformers`, the GPU model, memory, and driver from `nvidia-smi`, and the full server log. Measured runs are written to `experiments/` and committed; harness smoke tests are written to a separate, git-ignored directory and are never reported. Every table in this paper is generated by one script, `analysis/make_tables.py`, from those committed files.

**Validation.** The harness was exercised end to end on a laptop CPU with llama.cpp and a 0.5B model before any GPU run. That check, documented in the repository as a mechanics check rather than a result, led to two changes: a duplication metric based on shared prefix fraction rather than any shared character, and the reference-reproducibility check, after two clean seeded runs on that server disagreed at temperature 0.8.

# 5 Methodology

**Setup.** One AWS `g5.xlarge` instance (one NVIDIA A10G, 24 GB; 4 vCPUs), Llama 3.1 8B Instruct in bf16, vLLM in its own Python 3.12 environment with default engine flags except a 4,096-token context cap (Llama 3.1's 128k default does not fit a 24 GB KV cache; every request here is far shorter). Defaults include automatic prefix caching, which the server logs confirm was on (`enable_prefix_caching=True`). The machine image ships no CUDA toolkit, so FlashInfer's sampler, which compiles at startup, was replaced by PyTorch sampling (`VLLM_USE_FLASHINFER_SAMPLER=0`); this is recorded in every run log. Versions are in Table 1.

Table 1: Software and hardware, from `run.json`.

| Config | vLLM | torch | GPU |
|---|---|---|---|
| default | 0.31.0 | 2.13.0 | NVIDIA A10G (23028 MiB), driver 595.91.07 |
| no APC | 0.31.0 | 2.13.0 | NVIDIA A10G (23028 MiB), driver 595.91.07 |

16 measured runs, 80 trials, harness commit(s) `90da903`. In-flight chunks received after the kill: 0 at most, over all trials.

**Grid.** One fixed prompt (an explanation of write-ahead logging), `max_tokens`=256, per-request seed 1234. Strategy ∈ {naive retry, recompute} × temperature ∈ $\{0, 0.8\}$ × kill point ∈ {25, 128, 230} chunks (about 10%, 50%, and 90% of the budget), 5 trials each: 12 runs and 60 trials, produced by `TRIALS=5 scripts/run_matrix.sh vllm`. Requests are issued one at a time, so there is never more than one request in a batch.

**Ablation.** After the grid showed $G \neq G'$, we repeated the kill-at-128 configurations for both strategies and temperatures with automatic prefix caching disabled (`--no-enable-prefix-caching`), all else equal: 4 runs, 20 trials, labeled "no APC".

**Hypotheses.** Stated before the runs:

**H1** At temperature 0, naive retry yields full duplication in every trial and recompute yields $F = G$ in every trial.

**H2** At temperature 0.8 with a fixed seed, vLLM reproduces $G$ across clean runs; if so, naive retry still yields full duplication and recompute diverges after the kill point.

**H3** Recovery latency is dominated by server restart, not by prefill of the delivered text.

# 6 Results

**Failure-free output was not unique.** In all 12 runs of the default configuration, $G \neq G'$ (Table 2). At temperature 0 the two references agreed for the first 506 characters and then took different wording ("The log is scanned from the end…" versus "The log is scanned to determine…"). With a fixed seed at temperature 0.8 they diverged at character 9. Neither is noise: every run's $G$ was identical to every other run's $G$ at the same temperature, and likewise for $G'$. The output was a deterministic function of the server's history, not of the prompt alone. Within a run, all trials after the first produced identical text, so the 5 trials per configuration are repeated observations of at most two distinct server histories, not independent samples.

Table 2: Failure-free reproducibility: first request after server start ($G$) versus second ($G'$). LCP is their common prefix in characters. "No APC": automatic prefix caching disabled.

| Config | $T$ | $G$=$G'$ (runs) | LCP | $\lvert G\rvert/\lvert G'\rvert$ |
|---|---|---|---|---|
| default | 0 | 0/6 | 506 | 1284/1277 |
| default | 0.8 | 0/6 | 9 | 1219/1202 |
| no APC | 0 | 2/2 | 1228 | 1228/1228 |
| no APC | 0.8 | 2/2 | 1219 | 1219/1219 |

**Naive retry contradicted the client.** The restarted server is always in the state that produced $G$, and the replay equaled $G$ in all 30 naive-retry trials (Table 3). The text the client had already shown, $D$, came from a warm server and followed $G'$. Wherever the kill fell after the point at which $G$ and $G'$ part, the replay therefore contradicted $D$: in 25 of 30 trials, at

Table 3: Client-visible outcome per configuration (5 trials each), generated from the raw logs by `analysis/make_tables.py`. $|D|$: characters delivered before the kill (two values when trial 0 differed from trials 1–4). Contradicts $D$: the naive replay disagreed with text already shown. $F{=}G$, $F{=}G'$: the final text equals the first or second failure-free reference.

| Config | Strategy | $T$ | Kill (chunks) | $\lvert D\rvert$ (chars) | Contradicts $D$ | $F{=}G$ | $F{=}G'$ |
|---|---|---|---|---|---|---|---|
| default | naive | 0 | 25 | 113 | 0/5 | 5/5 | 0/5 |
| default | naive | 0 | 128 | 643 | 5/5 | 5/5 | 0/5 |
| default | naive | 0 | 230 | 1145 | 5/5 | 5/5 | 0/5 |
| default | naive | 0.8 | 25 | 105 | 5/5 | 5/5 | 0/5 |
| default | naive | 0.8 | 128 | 590 | 5/5 | 5/5 | 0/5 |
| default | naive | 0.8 | 230 | 1079 | 5/5 | 5/5 | 0/5 |
| default | recomp. | 0 | 25 | 113 | n/a | 0/5 | 0/5 |
| default | recomp. | 0 | 128 | 625/643 | n/a | 0/5 | 0/5 |
| default | recomp. | 0 | 230 | 1143/1145 | n/a | 0/5 | 1/5 |
| default | recomp. | 0.8 | 25 | 105/114 | n/a | 4/5 | 0/5 |
| default | recomp. | 0.8 | 128 | 556/590 | n/a | 0/5 | 0/5 |
| default | recomp. | 0.8 | 230 | 1079/1085 | n/a | 0/5 | 1/5 |
| no APC | naive | 0 | 128 | 612 | 0/5 | 5/5 | 5/5 |
| no APC | naive | 0.8 | 128 | 601 | 0/5 | 5/5 | 5/5 |
| no APC | recomp. | 0 | 128 | 612 | n/a | 0/5 | 0/5 |
| no APC | recomp. | 0.8 | 128 | 601 | n/a | 5/5 | 5/5 |

character 506 at temperature 0 and at character 9 at temperature 0.8. The only clean case was temperature 0 with the kill at 25 chunks (113 characters, before character 506), where the replay began with $D$ and the anomaly was pure, removable duplication. A client that drops the first $|D|$ characters of a replay, the usual deduplication, would in the other 25 trials have produced a spliced text that no failure-free run generated.

**Recompute kept the shown text but changed the answer.** Recompute never contradicted $D$, but its final text matched $G$ in 4 of 30 trials and $G'$ in 2 of 30. At temperature 0 with the kill at 25 chunks, all five final texts departed from both references at character 267, after the kill (113) and before $G$ and $G'$ part (506): a third greedy output. The same third point of departure appeared without any failure. In each recompute run, trials 1–4 ran on a server whose only earlier request was trial 0's recovery (prompt plus $D$), and their pre-kill streams, which no failure had yet touched, departed from both references in 16 of 24 trials. The two $F{=}G'$ cases are trial 0 at the 230-chunk kill; the four $F{=}G$ cases are trials 1–4 at temperature 0.8 with the kill at 25 chunks, where a 25-token recovery prefix left the continuation on the cold-server path.

**Hypotheses. H1 is refuted**: at temperature 0, naive retry gave full duplication only at the earliest kill point (5 of 15 trials) and recompute never reproduced $G$ (0 of 15). **H2's premise is refuted**: a fixed seed did not reproduce $G$ across requests, so its predictions cannot be tested as stated. **H3 is supported** (Table 4): restart took 108–195 s and the first recovered token arrived 0.23–0.32 s after the server was ready. Recompute's extra prefill was visible but small: median time to first token after restart rose from 0.24 to 0.27 s at temperature 0 and from 0.28 to 0.31 s at 0.8 as the kill moved from 25 to 230 chunks, against 0.23–0.28 s for naive retry. The higher median restart for recompute is not a strategy effect: restart happens before the recovery request is sent, and both initial startup and restart rose from about 110–122 s to 160–195 s from the third trial of the temperature-0, 230-chunk recompute run onward and stayed there for the rest of the grid; the cause was not identified. In the ablation, run later the same day, restarts were back to 107–120 s.

**Prefix-cache ablation.** With automatic prefix caching disabled (rows "no APC"), $G = G'$ in all 4 runs at both temperatures (Table 2), and naive retry no longer contradicted the client: in 10 of 10 trials the replay equaled both references and began with $D$, so the anomaly was pure duplication that a client can remove (Table 3). The cache setting also changed the reference itself at temperature 0: the no-APC output departs from the default configuration's $G$ at character 267, although $G$ is the first request after start, when the cache is empty. At temperature 0.8 the no-APC reference equals the default $G$ exactly.

Recompute behaved differently by temperature. At 0.8 its final text equaled $G$ in 5 of 5 trials. At temperature 0 it equaled neither reference in 5 of 5: all five departed at character 655, 43 characters after the kill (612), so recompute changed the greedy answer even with no prefix cache. That final text is character-for-

Table 4: Latency in seconds, median (min–max). Restart: kill to server ready. TTFT: server ready to first recovered chunk. Recovery: kill to first recovered chunk.

| Config | Strategy | Trials | Restart | TTFT after restart | Recovery |
|---|---|---|---|---|---|
| default | naive | 30 | 116.6 (108.1–122.1) | 0.26 (0.23–0.28) | 116.9 (108.3–122.4) |
| default | recomp. | 30 | 177.7 (110.1–195.2) | 0.27 (0.23–0.32) | 178.0 (110.4–195.5) |
| no APC | naive | 10 | 109.6 (107.1–120.1) | 0.25 (0.23–0.28) | 109.9 (107.3–120.4) |
| no APC | recomp. | 10 | 111.1 (108.1–120.1) | 0.28 (0.25–0.30) | 111.4 (108.4–120.4) |

character identical to the default configuration's recompute text at the 25-chunk kill. Recompute's answer at temperature 0 is therefore not explained by the cache. It is consistent with the delivered text being processed as prompt (prefill) rather than generated (decode), or with re-tokenization of $D$; these data do not separate the two.

# 7 Discussion

**Retry is not replay.** The usual argument for retrying an LLM request is that at temperature 0 it is idempotent up to duplication. On the default vLLM configuration measured here, it was not: the server that answers a retry is a fresh process, and a fresh process gave a different greedy answer from the warm one that had been streaming. For a chat client this means a visible rewrite of text already shown in 25 of 30 trials; for an agent that acted on a partial tool call or plan, it means the action was based on text that the final answer contradicts.

**What a client should do.** Within these measurements, recompute is the only client-side strategy that never contradicted what was shown, at the price of a final answer that no failure-free run produced, and a few tens of milliseconds of extra prefill. Naive retry with deduplication is unsafe unless the client verifies that the replay begins with $D$, and should replace rather than splice when it does not. An operator can remove the contradictions by disabling automatic prefix caching, as the ablation did; we did not measure the throughput cost, which on workloads with shared prefixes is the reason the cache exists.

**Why client-side recovery is not enough.** Even in the best case for recompute, the client cannot restore the server's state: neither the sampler's random state above temperature 0 nor, as measured here, whatever server state made $G$ differ from $G'$; and even with that state removed by disabling the prefix cache, recompute at temperature 0 did not reproduce the original continuation. Exactly-once delivery therefore requires state the client does not hold: a sequence-numbered token log, and the sampler state, committed before tokens are released, with recovery continuing from the log rather than regenerating. That protocol is the next stage of this project, and these measurements are its baseline.

# 8 Related work

**Fault-tolerant LLM serving.** DéjàVu [9] streams and replicates KV cache to recover from failures and reports server-side recovery time; it does not characterize what streaming clients receive. SpotServe [8] migrates in-flight context when preemptible instances are reclaimed with warning, and Llumnix [10] live-migrates requests for load balancing. Both handle planned moves; this study targets unplanned fail-stop crashes. **Serving engines.** Orca [12] introduced iteration-level scheduling, vLLM [7] block-managed KV memory, and SGLang [13] prefix sharing via RadixAttention. **Delivery semantics.** Chandy and Lamport [3] define consistent snapshots, Elnozahy et al. [5] output commit, and Flink [2] and Spark Structured Streaming [1] build exactly-once sinks on them. **Inference nondeterminism.** He et al. [6] show that LLM inference at temperature 0 is not reproducible across server load because kernels are not batch-invariant, and build batch-invariant kernels. Our observation is the fault-tolerance consequence: requests here were issued one at a time, yet output still depended on what the server had served before, so a retry after a crash is not a replay. Optimal checkpoint intervals [11, 4] motivate later stages of this project.

# 9 Threats to validity

- **Trials are not independent.** Output was deterministic given server history, so 5 trials per configuration are repeated observations of one or two histories. Counts such as "25 of 30" describe how many configurations and histories were affected, not a sampled rate.

- **One prompt, one model, one GPU.** The divergence positions (characters 506 and 9) are specific to this prompt and to Llama 3.1 8B on an A10G. Other prompts may diverge earlier, later, or not at

all within 256 tokens; tensor-parallel serving may differ.

- **One engine.** SGLang was planned as a second engine. On the machine image used (Ubuntu 26.04 Deep Learning AMI, no CUDA toolkit) it could not be started: several of its components compile GPU kernels at startup. The harness supports it unchanged, and the failed attempts are documented in the repository.
- **Sampler substitution.** vLLM ran with PyTorch sampling instead of FlashInfer's sampler, because of the missing CUDA toolkit. The temperature-0 results do not involve random sampling, but the 0.8 results may differ under FlashInfer.
- **Mechanism.** The ablation shows that, on this setup, the dependence on server history requires automatic prefix caching. It does not show which computation inside the cached path differs; kernels whose results depend on batch or prefill shape [6] are a plausible candidate that we did not test. The ablation covered only the 128-chunk kill point (20 trials).
- **Re-tokenization in recompute.** Recompute sends $D$ as text, which the engine re-tokenizes; a boundary that tokenizes differently can change the continuation. The recompute departures observed here were after the kill boundary (e.g. character 267 against a kill at 113).
- **Fail-stop only; latency includes restart.** `SIGKILL` is a clean crash; GPU hangs and silent corruption are not modeled. With a single instance, recovery includes model reload; a warm standby would be faster. Restart time also drifted during the grid (see H3).

## 10 Conclusion and future work

On vLLM with default settings, the failure-free output for a fixed prompt depended on what the server had served before, at temperature 0 as well as with a fixed seed. As a result, naive retry after a crash contradicted text the client had already shown in 25 of 30 trials, and recompute, while never contradicting it, produced a final text that no failure-free run produced in 24 of 30. Disabling automatic prefix caching made failure-free output reproducible and turned naive retry's contradictions into removable duplication, which shows that the cache is necessary for the history dependence on this setup. It did not make recompute exact at temperature 0.

Next stages: (0) repeat the grid on SGLang with a CUDA toolkit available, with more prompts, and with batch-invariant kernels [6]; (1) a gateway that assigns sequence numbers, logs tokens and sampler state before release, and continues from the log on recovery, evaluated on the same harness for zero anomalies and per-token overhead; (2) measured recompute-versus-checkpoint cost curves for the KV cache; (3) an adaptive per-request checkpoint policy.

**Artifact.** Code, raw logs, and the table-generating script are at `https://github.com/rmedipallycic/llm-inference-fault-tolerance` (Apache-2.0). Every number above traces to a committed log file.

## References


[1] Michael Armbrust, Tathagata Das, Joseph Torres, Burak Yavuz, Shixiong Zhu, Reynold Xin, Ali Ghodsi, Ion Stoica, and Matei Zaharia. Structured streaming: A declarative API for real-time applications in Apache Spark. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 601–613, 2018.

[2] Paris Carbone, Gyula Fóra, Stephan Ewen, Seif Haridi, and Kostas Tzoumas. Lightweight asynchronous snapshots for distributed dataflows, 2015. arXiv:1506.08603.

[3] K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985.

[4] J. T. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3):303–312, 2006.

[5] E. N. Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys*, 34(3):375–408, 2002.

[6] Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism, 2025. `https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/`.

[7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[8] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. SpotServe: Serving generative large language models on preemptible instances. In *Proceedings of ASPLOS*, pages 1112–1127, 2024.

[9] Foteini Strati, Sara McAllister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. DéjàVu: KV-cache streaming for fast, fault-tolerant generative LLM serving. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 46745–46771, 2024.

[10] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[11] John W. Young. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9):530–531, 1974.

[12] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 521–538, 2022.

[13] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2312.07104.